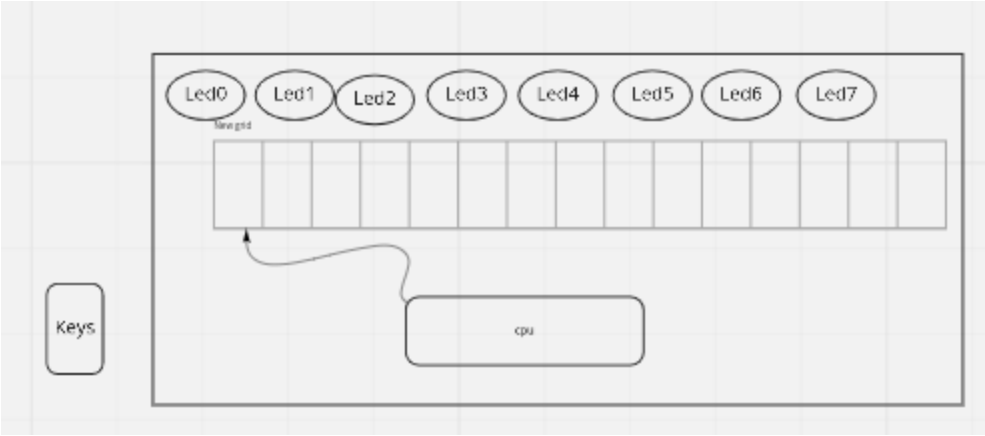
Operating Systems

Ahmad Yoosofan

University of Kashan

https://yoosofan.github.io/en/

Yoosofan Imaginary Computer
Based on Morris Mano's famous book


Led0
Led1
Led2
Led3
Led4
Led5
Led6
Led7
Keys
cpu

Instruction Set(I)

```
AND: Logical AND memory with AC
ADD: Arithmetic ADD memory with AC
LDA: Load from memory to AC
STA: Store AC to memory
BUN: Branch unconditional
ISZ: Increment and skip if zero
CLA: Clear AC
CLE: Clear E
CMA: Complement AC
CME: Complement E
CIR: Circulate right (AC and E)
CIL: Circulate left (AC and E)
```

```
INC:    Increment AC
SPA:    Skip if positive AC
SNA:    Skip if negative AC
SZA:    Skip if zero AC
SZE:    Skip if zero E
HLT:    Halt
OUT:    Output a character from AC
SKO:    Skip if output flag
NOP:    No operation
```

Instruction Set Bianary(I)

AND:	00001
ADD:	00010
LDA:	00011
STA:	00100
BUN:	00101
ISZ:	00110
CLA:	00111
CLE:	01000
CMA:	01001
CME:	01010
CIR:	01011
CIL:	01100

INC:	01101
SPA:	01110
SNA:	01111
SZA:	10000
SZE:	10001
HLT:	10010
OUT:	10011
SKO:	10100
NOP:	10101

hex pad connect to microcontroller

https://www.circuitstoday.com/interfacing-hex-keypad-to-8051

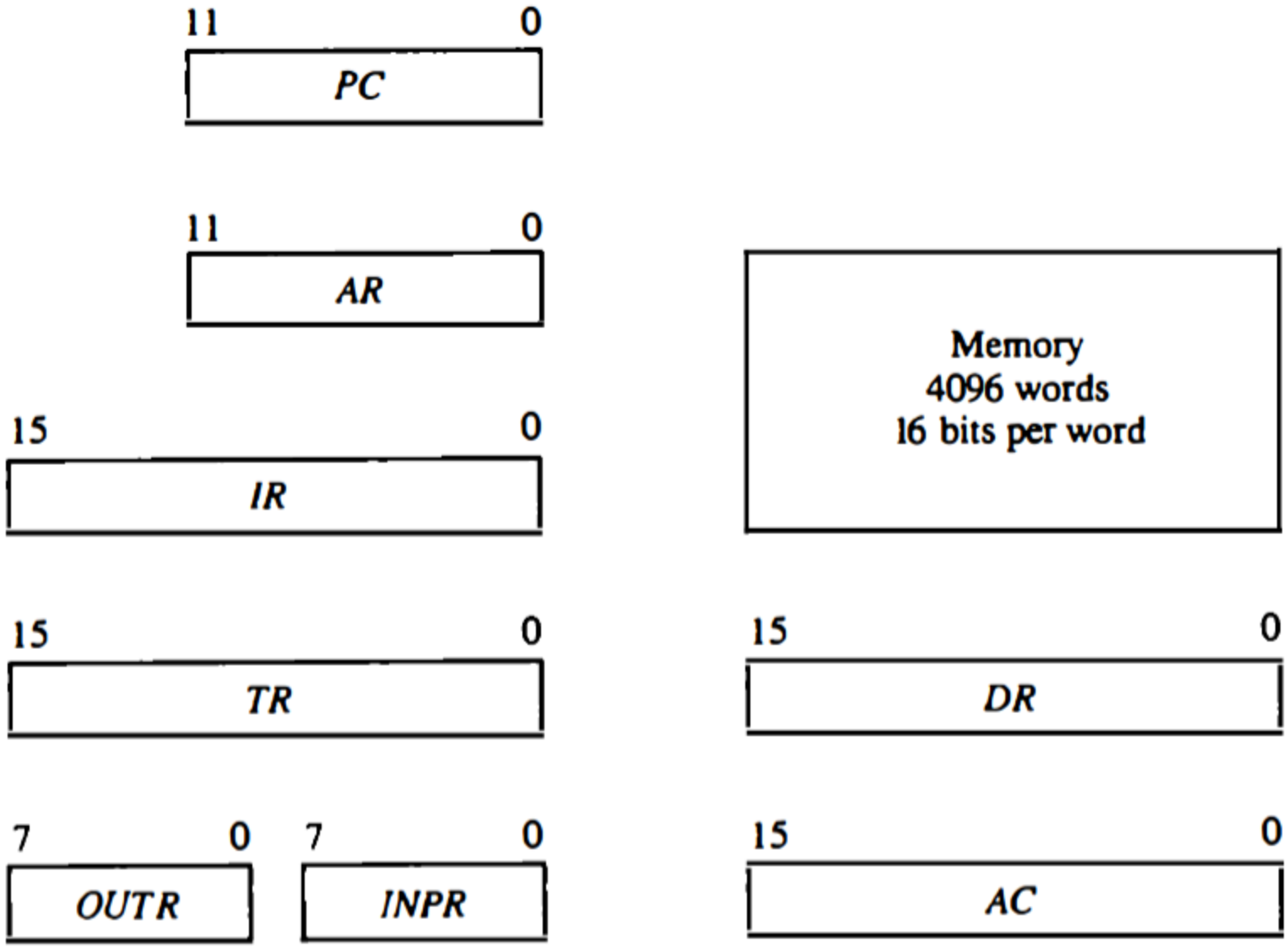
https://circuitdigest.com/microcontroller-projects/keypad-interfacing-with-avr-atmega32

Example

- https://github.com/yoosofan/mano-computer-simulator-js
- https://yoosofan.github.io/mano-computer-simulator-js/
- https://github.com/Naheel-Azawy/Simple-Computer-Simulator/blob/master/test/test-symbolic
- https://github.com/Naheel-Azawy/Simple-Computer-Simulator/blob/master/test/test
- https://github.com/Naheel-Azawy/Simple-Computer-Simulator/tree/master/test

Other assembly

- http://imrannazar.com/arm-opcode-map
- https://iitd-plos.github.io/col718/ref/arm-instructionset.pdf
- https://wiki.osdev.org/X86-64_Instruction_Encoding
- https://cs.brown.edu/courses/cs033/docs/guides/x64_cheatsheet.pdf
- https://sites.google.com/site/nttrungmtwiki/home/rce/assembly-language/x64-opcode-and-instruction-reference-home
- http://ref.x86asm.net/coder64.html
- arm 32 opcodes
- http://z80-heaven.wikidot.com/instructions-set:ld
- http://z80-heaven.wikidot.com/opcode-reference-chart
- https://smallcomputercentral.files.wordpress.com/2017/12/asm80-com-tutorial-e1-0-01.pdf
- https://stackoverflow.com/questions/22838444/convert-an-8bit-number-to-hex-in-z80-assembler
- https://www.vcfed.org/forum/forum/technical-support/vintage-computer-programming/76419-z80-hello-world-example-in-hex
- https://www.cemetech.net/forum/viewtopic.php?t=15710&start=0
- z80 assembly codes


11
0
PC
11
0
AR
15
0
IR
15
0
TR
7
0
OUTR
7
0
INPR
Memory
4096 words
16 bits per word
15
0
DR
15
0
AC

15	14 … 12	11 … 0
I	Opcode	Address

(Opcode = 000 through 110)

(a) Memory – reference instruction

15 … 12	11 … 0
0 1 1 1	Register operation

(Opcode = 111, $I = 0$)

(b) Register – reference instruction

15 … 12	11 … 0
1 1 1 1	I/0 operation

(Opcode = 111, $I = 1$)

(c) Input – output instruction

```
00101 00000  1010
00110 00000  1100
00111 00000  1110
01000

00000
```

اگر حداکثر ۳۲ دستور داشته باشیم پس پنج بیت برای دستورها نیاز داریم برای سادگی فرض می‌کنیم که طول همهٔ دستورها یکسان است یعنی هم دو بایت را می‌گیرند فرض کنید دستورها پنج بیت نیاز دارند پس ۱۱ بیت برای آدرس

حداکثر حافظهٔ این کامپیوتر چقدر می‌تواند باشد. اگر بخواهیم بایتی آدرس دهی کنیم

$$۲^{۱۱} = ۲kB$$

B = Byte

اگر آدرس‌دهی را دو بایتی در نظر بگیریم

$$۴kB \text{ (word = 2 byte)}$$

Output

LED

seven segment

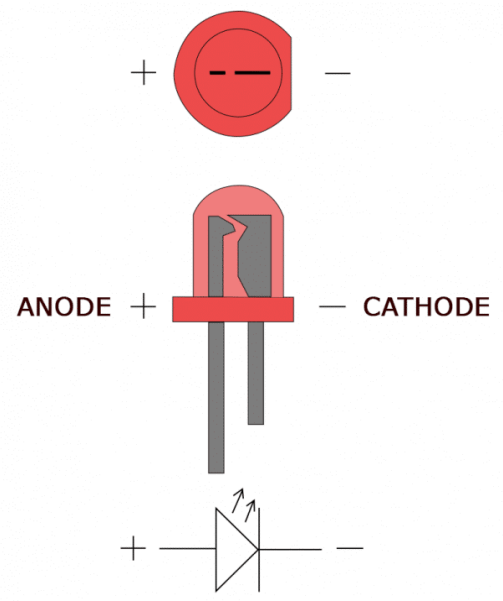
- https://www.rapidtables.com/convert/number/hex-to-binary.html
- convert hex to binary
- https://clrhome.org/asm/

Output Problem

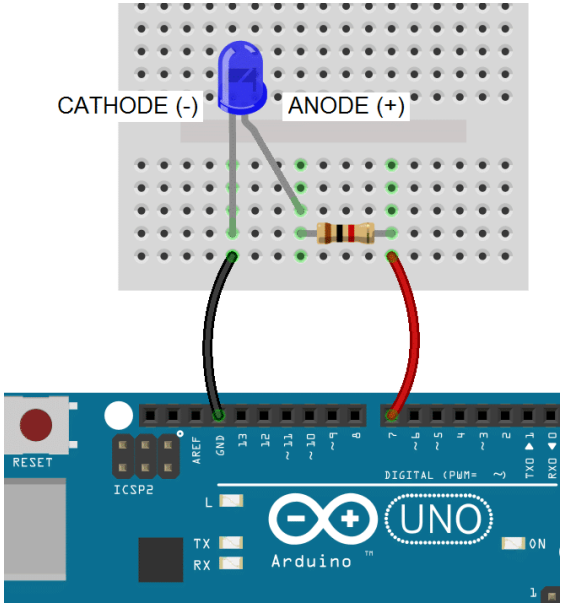
```
lda a
add b
sta c
out
hlt
a, 5
b, 2
c, 0
```

```
..........
..........

LB1: out
     sko
     bun LB1

...........
...........
```

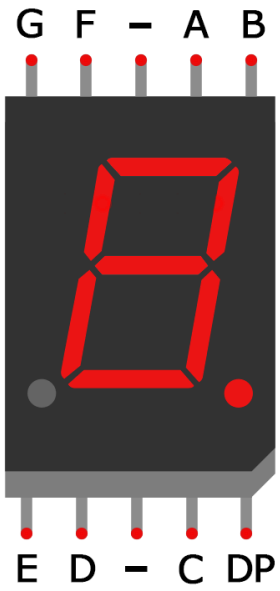
Simple LED

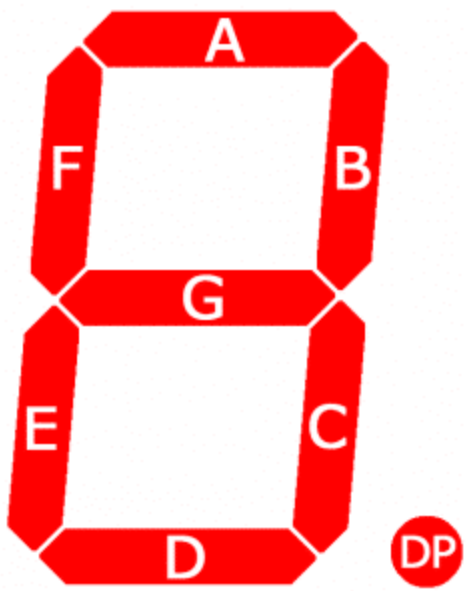


circuitbasics



Seven segment display





circuitbasics askingthelot

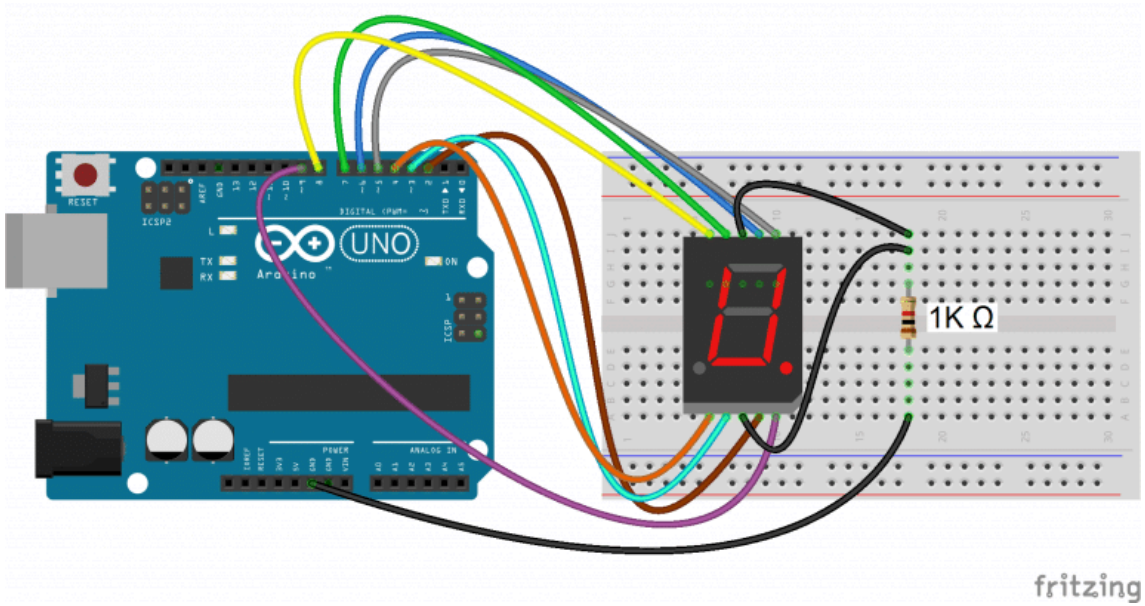
youtu.be element14

Imaginary Computer

- Consider it as real a computer
- Think about business plan
- Consider customers' need
- Consider other companies

YIC 30

Seven segment

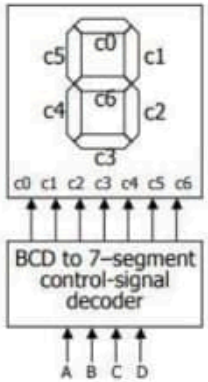


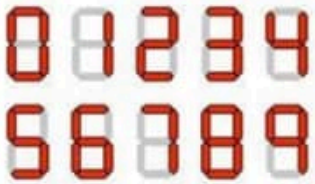
circuitbasics

Issues of YIC 30

- Convert binary number to 7 segment code
- Old codes only LED
- LED & seven segment
- Changing CPU
- Cost of changes
- Just one 7 segment ?
- for every output, seven segment code should be added

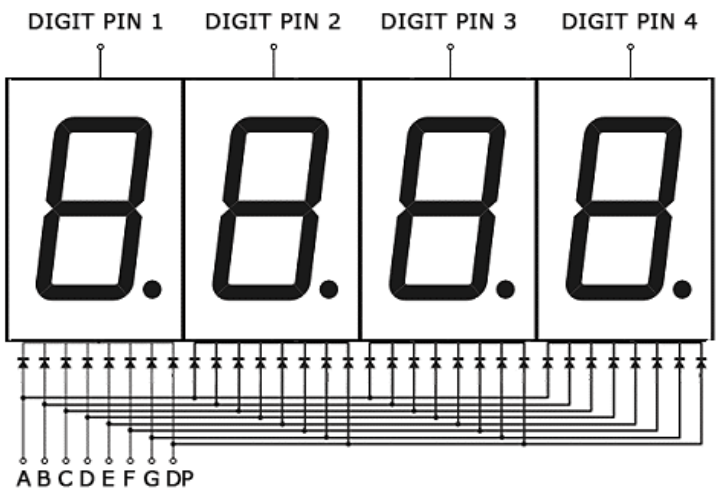
Hardware insead of Software

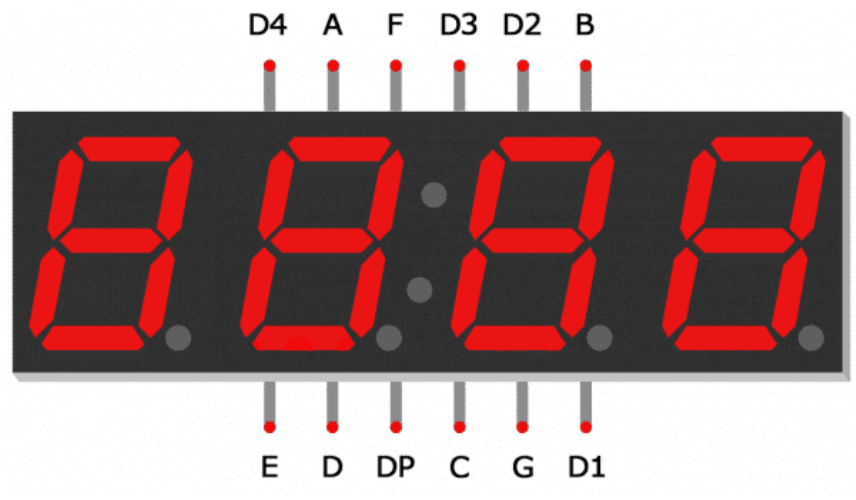




A	B	C	D	C0	C1	C2	C3	C4	C5	C6
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	0	0	1	1
1	0	1	–	–	–	–	–	–	–	–
1	1	–	–	–	–	–	–	–	–	–

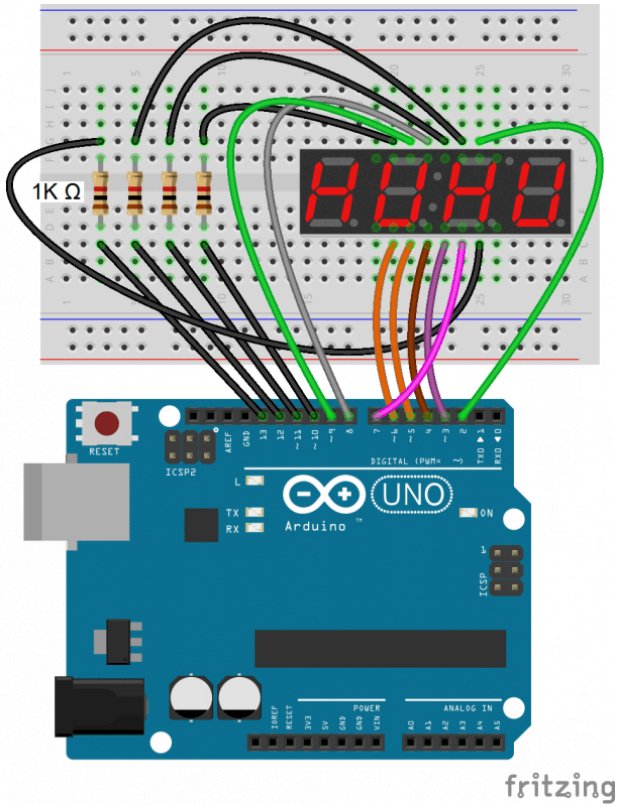
4 Digit 7-Segment Displays





circuitbasics

Connecting 4 Digit 7-Segment Displays



Arduino Print 4 to 7-segment

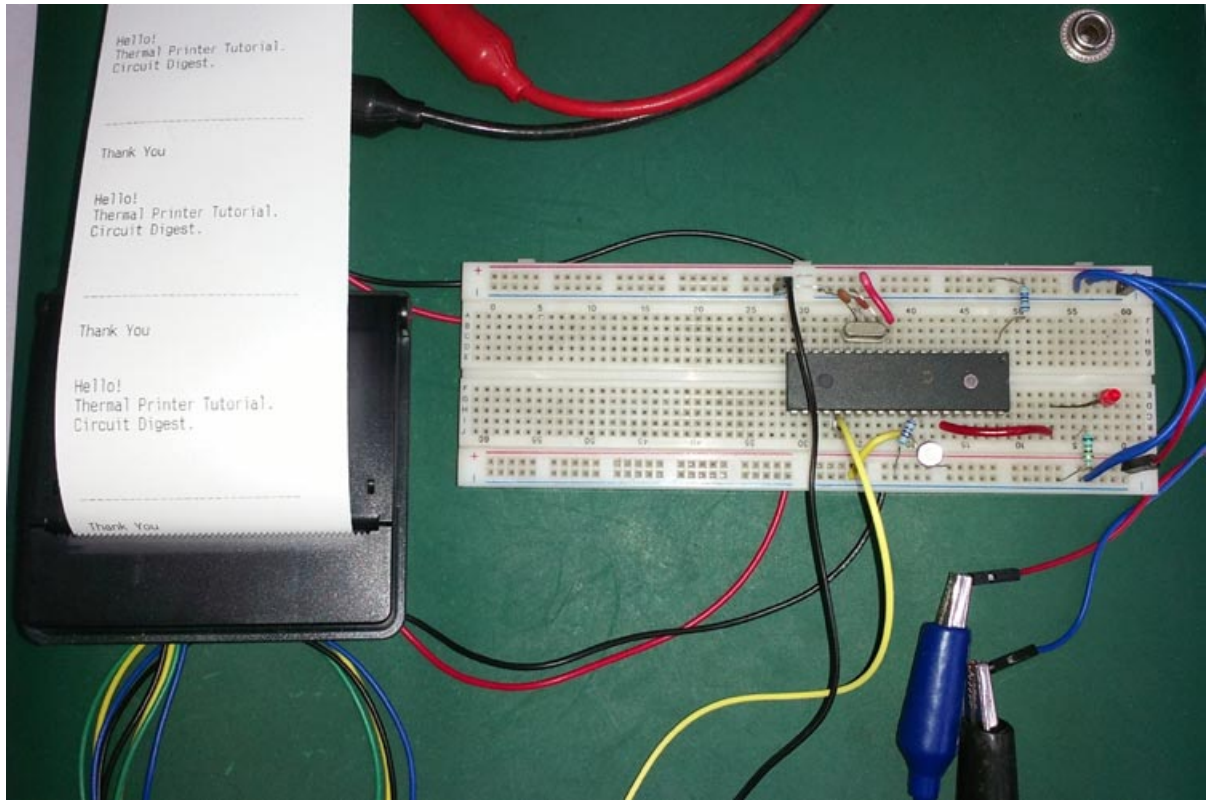
```
#include "SevSeg.h"
SevSeg sevseg;

void setup(){
  byte numDigits = 1;
  byte digitPins[] = {};
  byte segmentPins[] =
    {6, 5, 2, 3, 4, 7, 8, 9};
  bool resistorsOnSegments = true;

  byte hardwareConfig = COMMON_CATHODE;
  sevseg.begin(hardwareConfig,
    numDigits, digitPins, segmentPins,
    resistorsOnSegments
  );
  sevseg.setBrightness(90);
}

void loop(){
  sevseg.setNumber(4);
  sevseg.refreshDisplay();
}
```

Segment Pin	Arduino Pin
A	6
B	5
C	2
D	3
E	4
F	7
G	8
DP	9

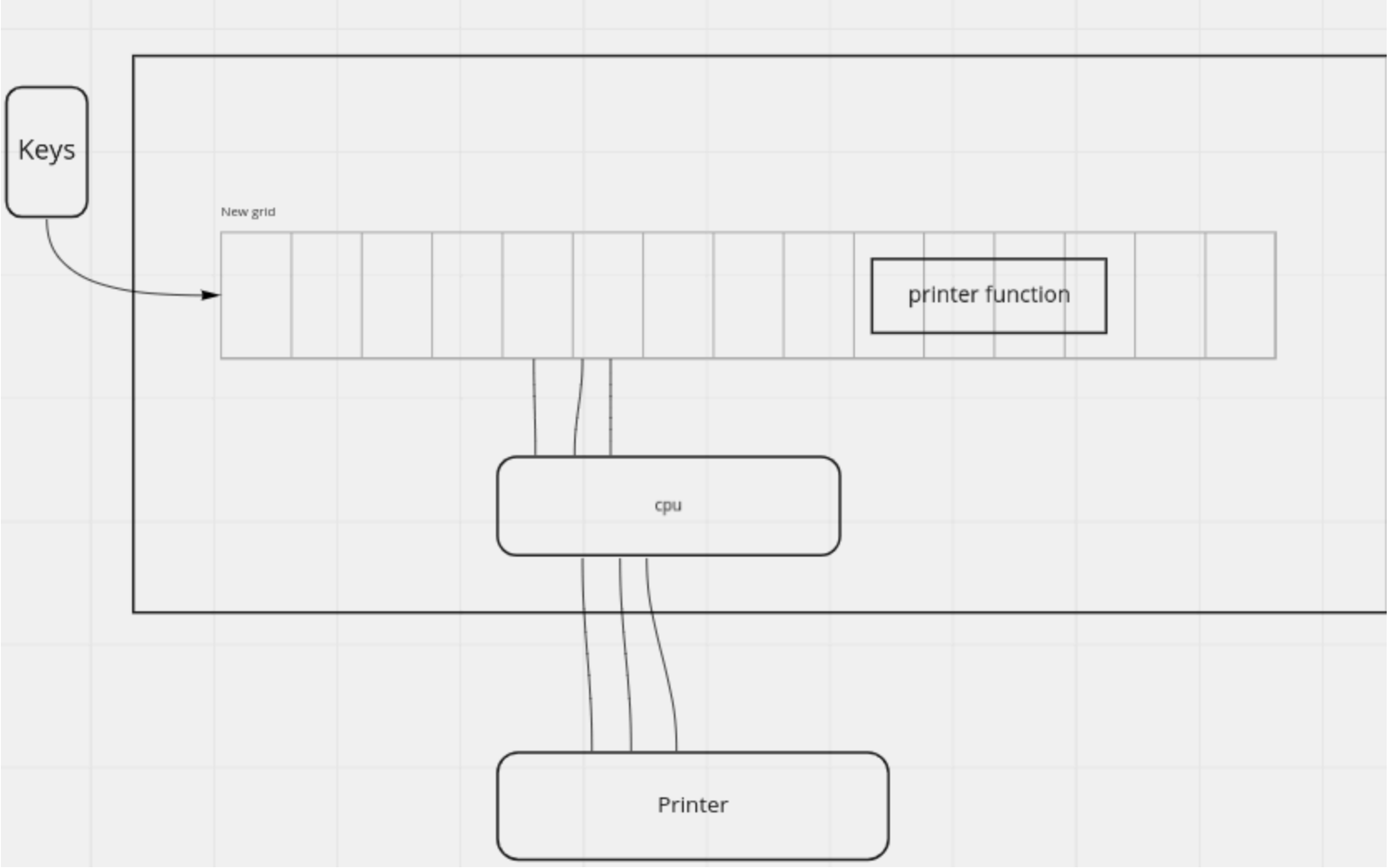
circuitbasics

YIC 40 - BSA

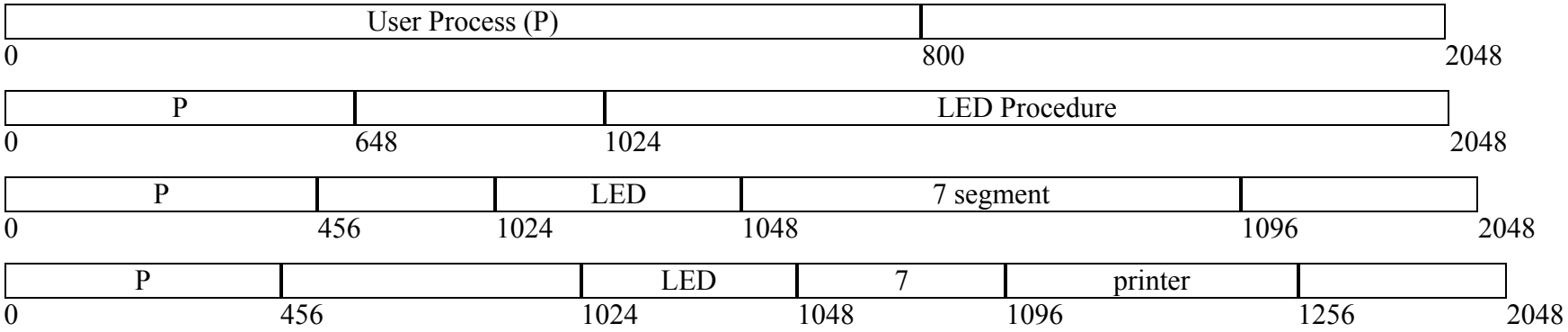
1. LED output code
2. 7 segment code
3. Printer
4. output selector
5. Adding porecedures
6. Device Drivers
7. Adding more devices
8. No error checking
9. Send data
10. API (protocol)



Printer Routine

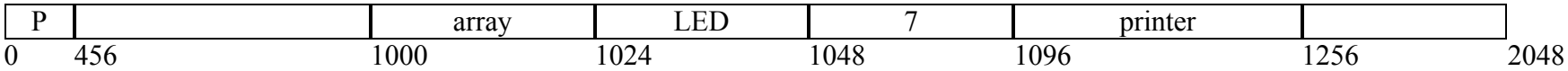


Adding procedurs to memory

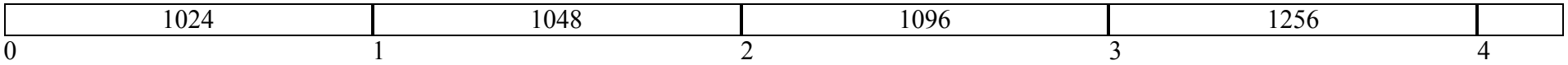


Users (programmers) should know where these precedures are

YIC50 - Array of Addresses



Array



indirect BSA

User chooses between old cpu or new one

YIC60 - Input Device (Card Reader)

1. Card Reader
2. Necessary Loops
3. Check Errors
4. Polling Method

- Hollerith and IBM keypunches, 1890
- IBM 011 Electric Key Punch(1923)
- IBM Type 032 Printing Punch(1935)
- A Key Punch Room in the 1960s

card punch

card reader

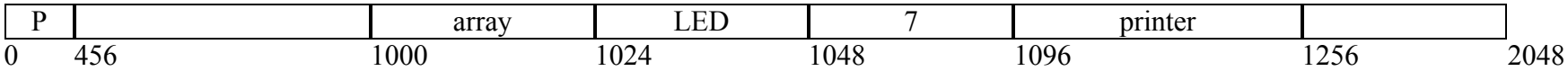
```
        ORG 0
START,  BSA READ1
        STA BYTE1
        LDA BYTE1
        BSZ OUTCH
        HLT

READ1,  HEX 0
RDCNT,  SKI
        BUN RDCNT
        INP
        BUN (READ1)

OUTCH,  HEX 0
OUTPR,  SKO
        BUN OUTPR
        OUT
        BUN (OUTCH)

BYTE1,  DEC     0
        END
```

Simple Input Output Code - YIC60

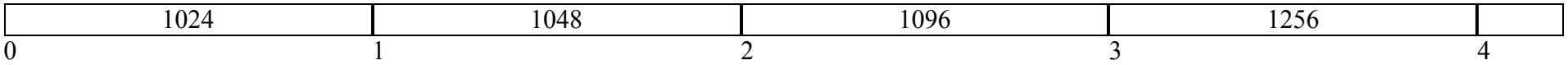
```
        ORG     0
START,  BSA     INPUT
        STA     NUM1

        BSA     INPUT
        STA     NUM2

        LDA     NUM1
        ADD     NUM2
         STA     RESULT

         BSA     OUTPUT

         HLT
```

```
INPUT,  HEX     0
POLL,   SKI
        BUN     POLL
        INP
        BUN     (INPUT)

OUTPUT, HEX     0
OUT_P,  SKO
        BUN     OUT_P
        OUT
        BUN     (OUTPUT)

// Data Section
NUM1,   DEC     0
NUM2,   DEC     0
RESULT, DEC     0

        END
```

YIC60 - Array of Addresses



Array



YIC60 code with array(I)

```
START   BSA     (PVT_INPUT)
        STA     NUM1
        BSA     (PVT_INPUT)
        STA     NUM2
        LDA     NUM1
        ADD     NUM2
        STA     RESULT
        BSA     (PVT_OUTPUT)
        HLT
NUM1,   DEC     0
NUM2,   DEC     0
RESULT, DEC     0

        ORG     150
PVT_INPUT,  HEX 200
PVT_OUTPUT, HEX 220

        ORG     200
INPUT,  HEX     0
IN_POLL,SKI
        BUN     IN_POLL
        INP
        BUN     (INPUT)
```

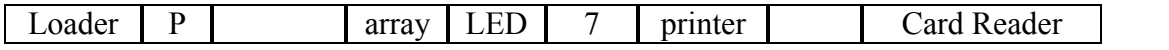
```
        ORG     220
OUTPUT, HEX     0
        STA     TEMP_OUT

OUT_POLL,SKO
        BUN     OUT_POLL
        LDA     TEMP_OUT
        OUT
        BSA     (PVT_DELAY)
        BUN     (OUTPUT)

        ORG     240
DELAY,  HEX     0
        LDA     DELAY_COUNT
        STA     DELAY_CTR

DELAY_LOOP,
        LDA     DELAY_CTR
```

YIC60 code with array(II)

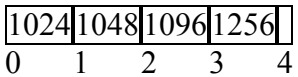
```
DELAY,  HEX     0
        LDA     DELAY_COUNT
        STA     DELAY_CTR

DELAY_LOOP,
        LDA     DELAY_CTR
        SZA
        BUN     CONTINUE_DELAY
        BUN     (DELAY)

CONTINUE_DELAY,
        DEC
        STA     DELAY_CTR
        BUN     DELAY_LOOP

        ORG     300
TEMP_OUT,   DEC     0
DELAY_COUNT,DEC     10
```

```
DELAY_CTR,  DEC     0
PVT_DELAY,  HEX     52
        END
```

YIC70 - Adding Loader



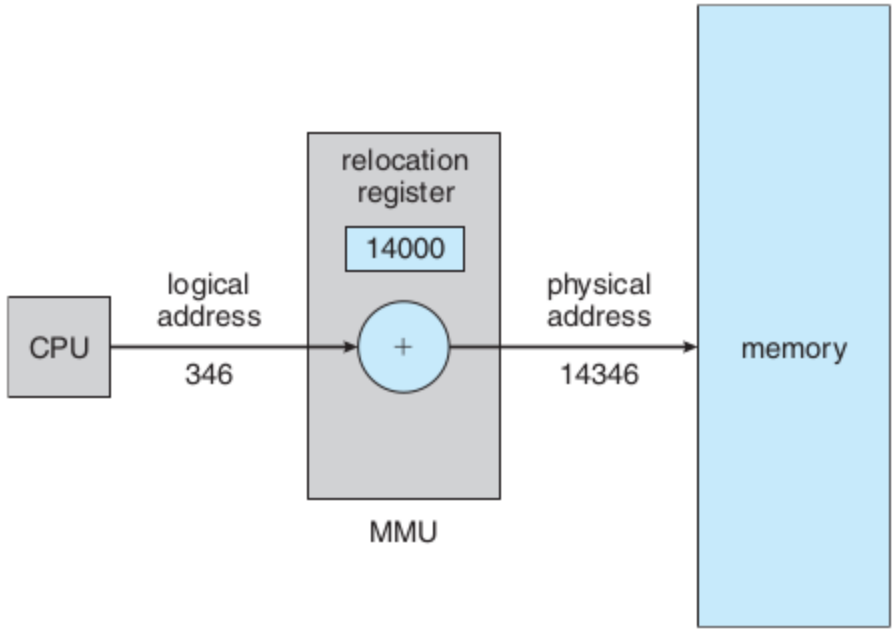
Array



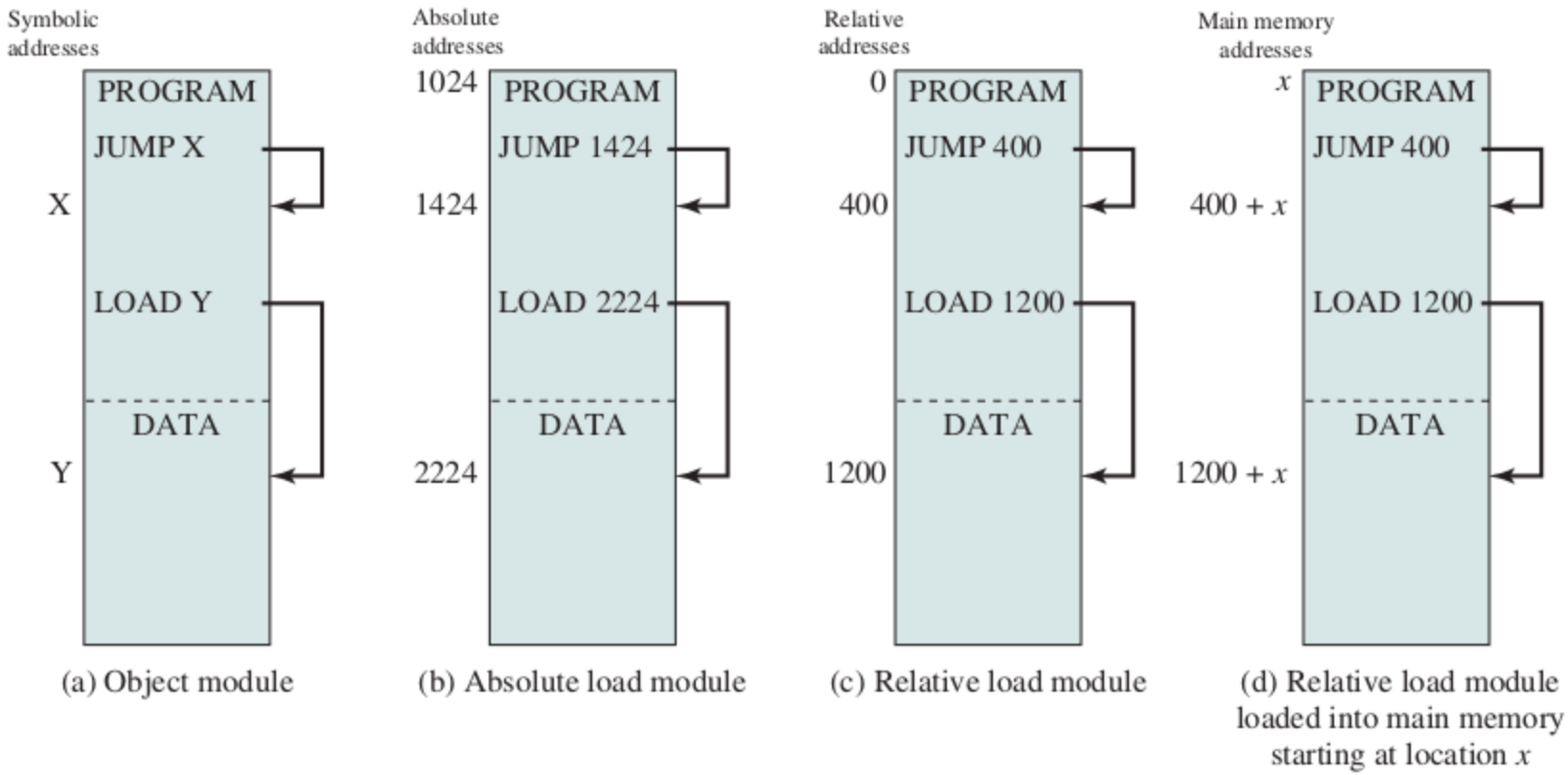
```
START,  BSA     READ_COUNT
        STA     BYTE_COUNT
        BSA     INIT_LOAD
        BSA     LOAD_LOOP
        BSA     EXECUTE
        BUN     0
        HLT

READ_COUNT, HEX 0
RD_CNT, SKI
        BUN     RD_CNT
        INP
        BUN     READ_COUNT I

INIT_LOAD, HEX 0
        LDA     ZERO
        STA     LOAD_PTR
        STA     CURRENT_IDX
        BUN     INIT_LOAD I

LOAD_LOOP, HEX 0
        LDA     CURRENT_IDX
        SUB     BYTE_COUNT
        SPA
        BUN     LOAD_BYTE
        BUN     LOAD_LOOP I

LOAD_BYTE,
        BSA     READ_BYTE
        STA     TEMP_BYTE

        LDA     LOAD_PTR
        STA     STORE_PTR
        LDA     TEMP_BYTE
        STA     STORE_PTR I

        LDA     LOAD_PTR
        INC
        STA     LOAD_PTR
```

```
        STA     LOAD_PTR
        LDA     CURRENT_IDX
        INC
        STA     CURRENT_IDX
        BUN     LOAD_LOOP

READ_BYTE, HEX 0
RD_BYT, SKI
        BUN     RD_BYT
         INP
         BUN     READ_BYTE I

         ORG     128
EXECUTE, HEX 0

         LDA     ZERO
         STA     RESULT
         BUN     EXECUTE I

         ORG     64
ZERO,       DEC     0
BYTE_COUNT, DEC     0
LOAD_PTR,   HEX     128
CURRENT_IDX,DEC     0
TEMP_BYTE,  DEC     0
STORE_PTR,  HEX     0
RESULT,     DEC     0

        END
```

YIC75 Relative Address

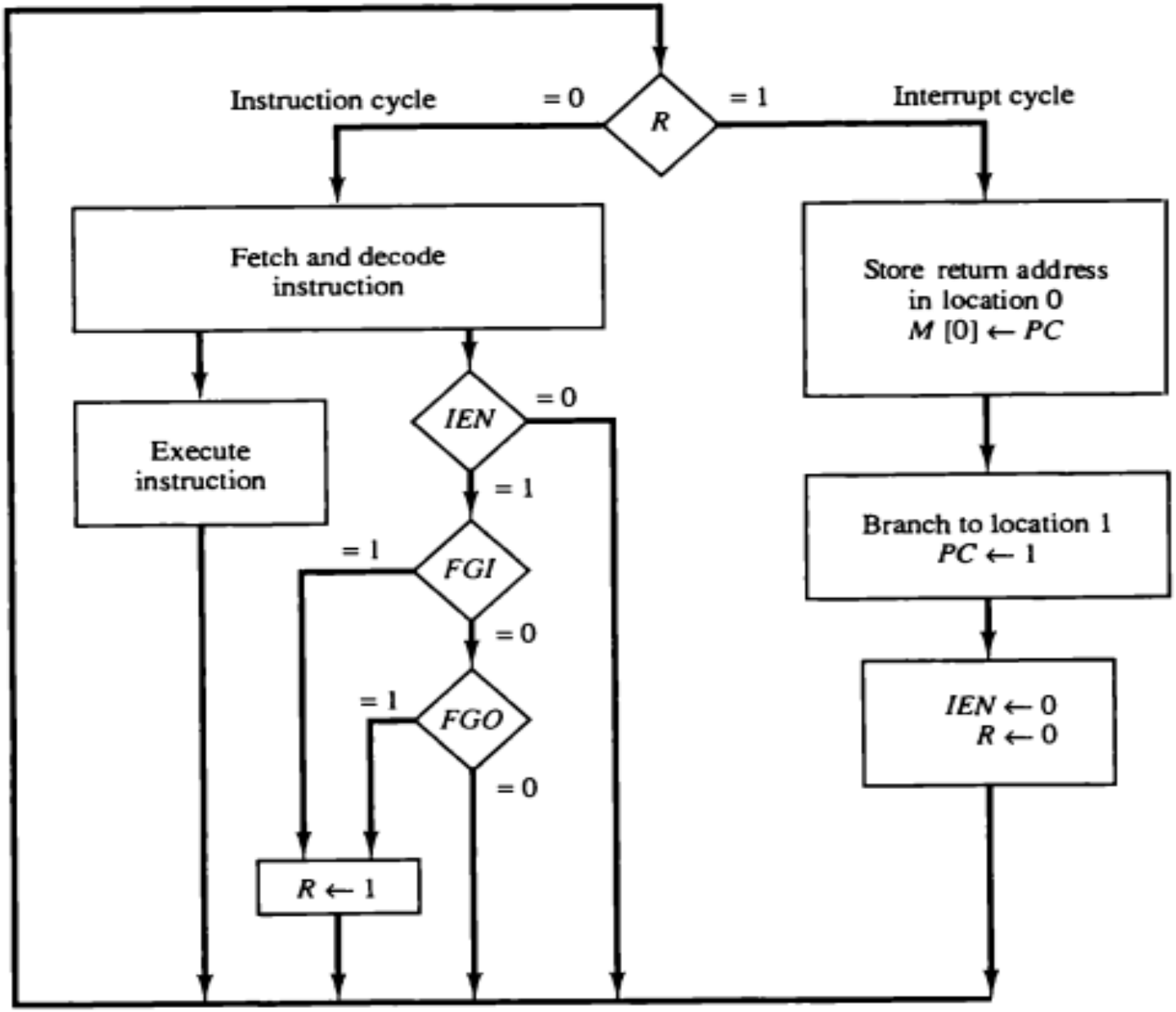


address binding, absolute and relocate loader



(a) Object module (b) Absolute load module (c) Relative load module (d) Relative load module loaded into main memory starting at location x

YIC80 - Interrupt



Interrupt-Driven Program

```
        BUN     MAIN
        BUN     ISR

MAIN,   LDA     ZERO
        STA     COUNT
        ION

WORK,   LDA     COUNT
        INC
         STA     COUNT
         BUN     WORK

ISR,     STA     SAVE
         BSA     IO
         ION
         LDA     SAVE
         BUN     0  I
```

```
IO,     HEX     0
        SKI
        BUN     OUTPUT
        INP
        STA     BUFFER
        BUN     IO  I
OUTPUT, SKO
        BUN     IO  I
        OUT
        BUN     IO  I

ZERO,   DEC     0
COUNT,  DEC     0
SAVE,   DEC     0
BUFFER, DEC     0
        END
```

Loader with interrupt (bootstrap)

```
# Advanced: Buffered Input with BSA Subroutines
src/in/Interrupt_Driven_Program_with_BSA_Subroutines_Advanced_with_buffer.asm
src/in/Interrupt_Driven_Program_with_BSA_Subroutines_Advanced_with_buffer_comments.asm

# Enhanced bootstrap loader with error checking
src/in/Bootstrap_Loader_Program_More_Robust_Version_with_Error_Checking.asm

# Relocating Bootstrap Loader with Base Register
src/in/loader4_base_register_comments.asm


# Simple Bootstrap Loader with Absolute Addressing
src/in/loader7_interrupt_Simple_Bootstrap_Loader_with_Absolute_Addressing.asm
src/in/loader7_interrupt_Simple_Bootstrap_Loader_with_Absolute_Addressing_comments.asm

src/in/loader7_user_program_comments.asm
src/in/loader7_user_program.asm


# Uses interrupt-driven I/O instead of polling
src/in/loader10_interrupt.asm
src/in/loader10_interrupt_comments.asm

# Interrupt-driven program that gets loaded
src/in/loader10_loaded_program.asm
src/in/loader10_loaded_program_comments.asm
```

```
  ORG 0
  BUN LOADER
  BUN ISR
  ORG 100
LOADER,LDA ZERO
  STA BYTES_LOADED
  STA PROG_SIZE
  STA INP_BUF
W4SIZE,LDA INP_BUF
  SZA
  BUN STORE_SIZE
  BUN W4SIZE
STORE_SIZE,
  LDA INP_BUF
  STA PROG_SIZE
  STA INP_BUF
  BSA LOAD_PROGRAM
  ION
  BSA USER_PROG I
  BUN LOADER

ISR,STA SAVE_AC
  SKI
  BUN CHECK_OUTPUT
  INP
  STA INP_BUF
  BUN ISR_EXIT
CHECK_OUTPUT,
  SKO
  BUN ISR_EXIT
  LDA OUTPUT_PENDING
  SZA
  BUN HAS_PENDING_OUTPUT
  BUN ISR_EXIT
HAS_PENDING_OUTPUT,
  LDA OUTPUT_DATA
  OUT
  LDA ZERO
  STA OUTPUT_PENDING
ISR_EXIT,
  LDA SAVE_AC
  BUN INDIRECT
SYS_WRITE_HANDLER, HEX 0
  STA OUTPUT_DATA
  LDA ONE
  STA OUTPUT_PENDING
  SKO
  BUN DEVICE_READY
  BUN SYS_WRITE_RETURN
DEVICE_READY,
  LDA OUTPUT_DATA
  OUT
  LDA ZERO
  STA OUTPUT_PENDING
SYS_WRITE_RETURN,
  BUN SYS_WRITE_HANDLER I

LOAD_PROGRAM, HEX 0
  LDA ZERO
  STA LOAD_INDEX
LOAD_LOOP,
  LDA LOAD_INDEX
  SUB PROG_SIZE
  SPA
  BUN WAIT_FOR_BYTE
  BUN LOAD_DONE
WAIT_FOR_BYTE,
  LDA INP_BUF
  SZA
  BUN STORE_BYTE
  BUN WAIT_FOR_BYTE
STORE_BYTE,
  LDA LOAD_INDEX
  ADD USER_PROG
  STA STORE_ADDR
  LDA INP_BUF
  STA STORE_ADDR I
  STA INP_BUF
  LDA LOAD_INDEX
  INC
  STA LOAD_INDEX
  BUN LOAD_LOOP
LOAD_DONE,
  BUN LOAD_PROGRAM I

USER_PROG, HEX 500
ZERO,      DEC 0
ONE,       DEC 1
PROG_SIZE, DEC 0
LOAD_INDEX,DEC 0
```

Interrupt and Relative Address Problem

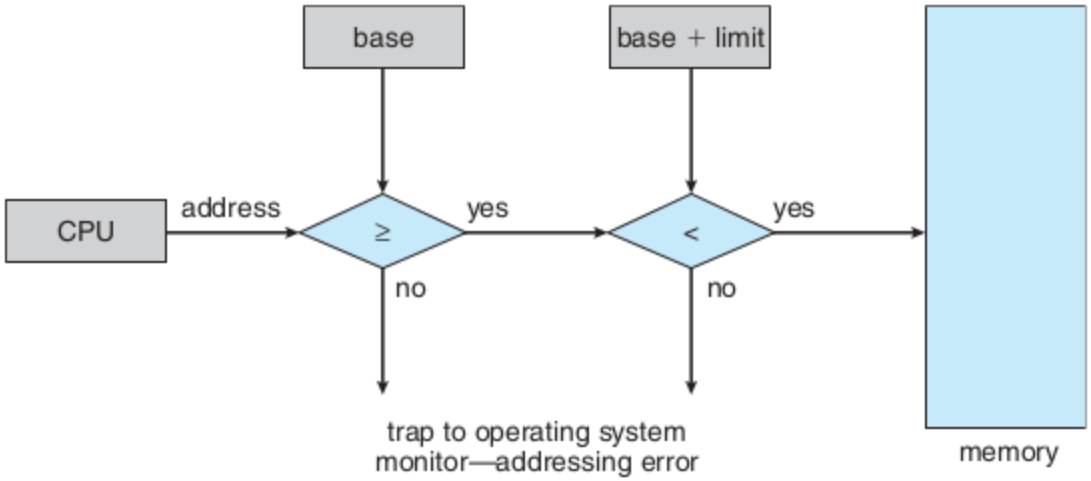
- Normal execution
- Interrupt time
- ISR needs

    1. predictable, fixed addresses
    2. resources across all processes
    3. Safety

Solution Dual Address Spaces

1. User mode (MOD 1)
2. Kernel mode (MOD 0)

YIC90 - Memory and CPU Protection

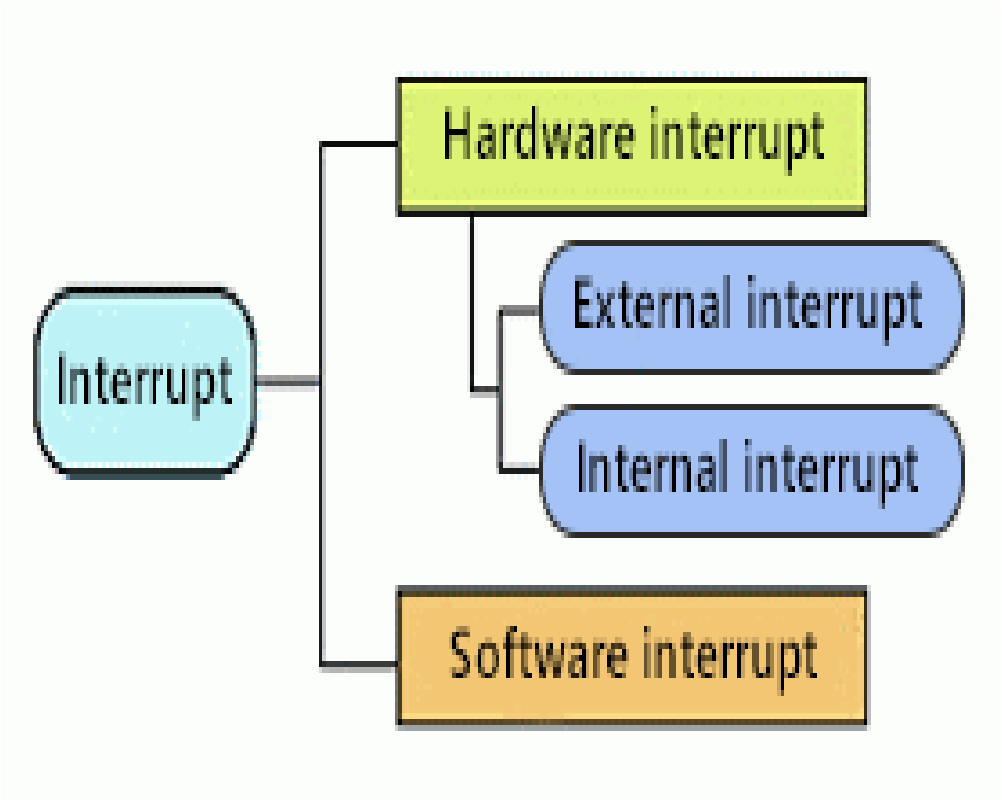


1. SKT like SKI and SKO
2. System Call ?
3. Change registers by the running process

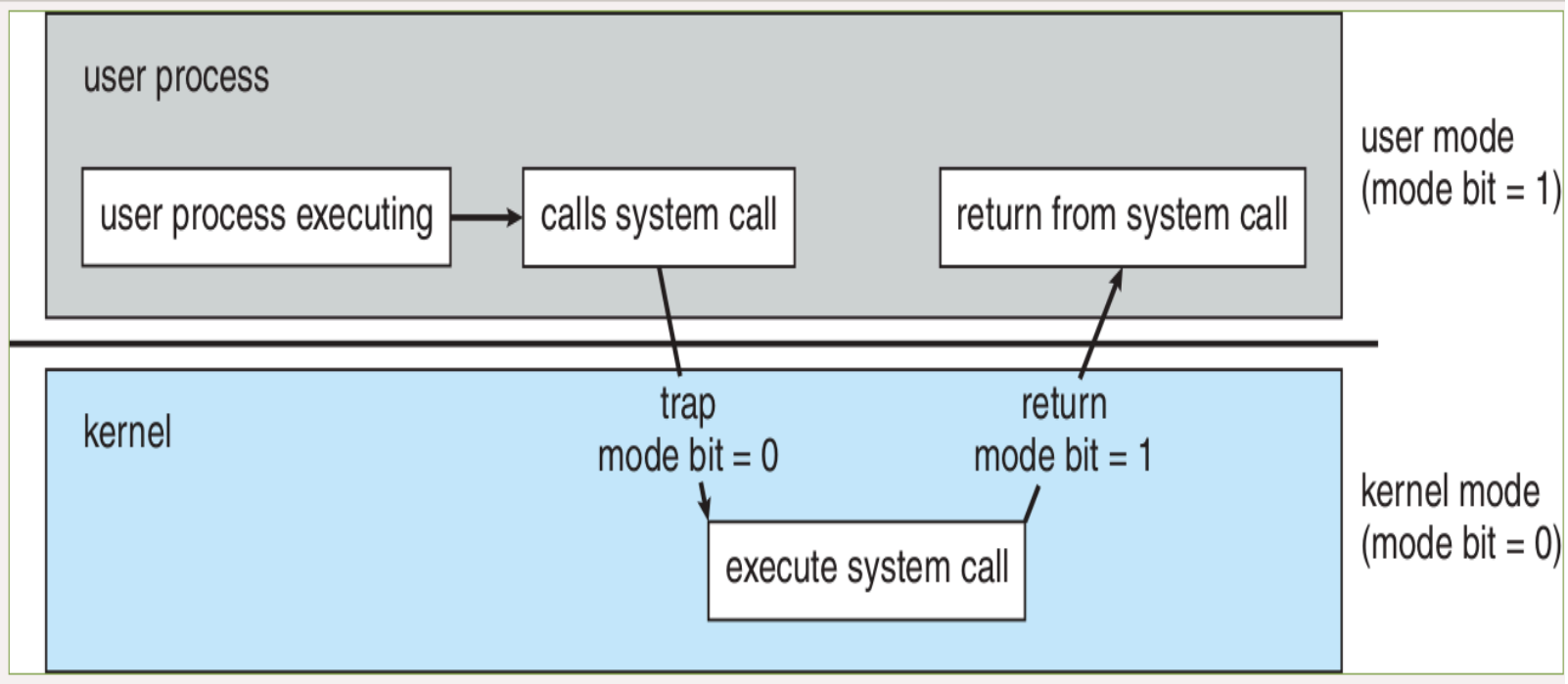
Software Interrupt

```
ISR,     STA     SAVE
         BSA     IO
         ION
         LDA     SAVE
         BUN     0  I

IO,      HEX     0
         SKI
         BUN     OUTPUT
         INP
         STA     BUFFER
         BUN     IO  I
OUTPUT,  SKO
         BUN     TRAP
         OUT
TRAP,    SKT
         BUN     IO  I
         BUN     100
```

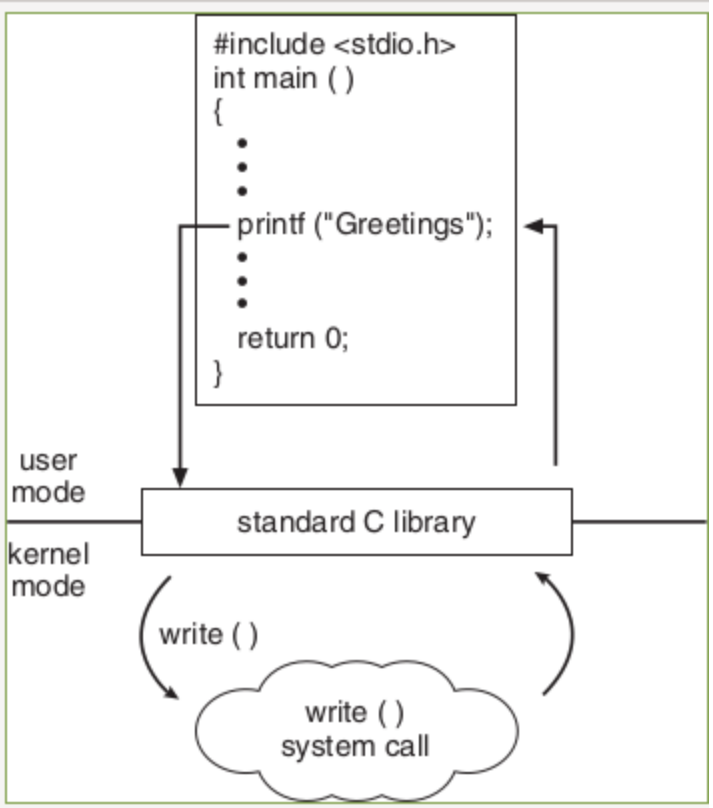
```
mov ah, 0x0e
; function number = 0Eh
; : Display Character
mov al, '!'
; AL = code of character
; to display
int 0x10
; call INT 10h,
; BIOS video service
```


Interrupt
Hardware interrupt
External interrupt
Internal interrupt
Software interrupt

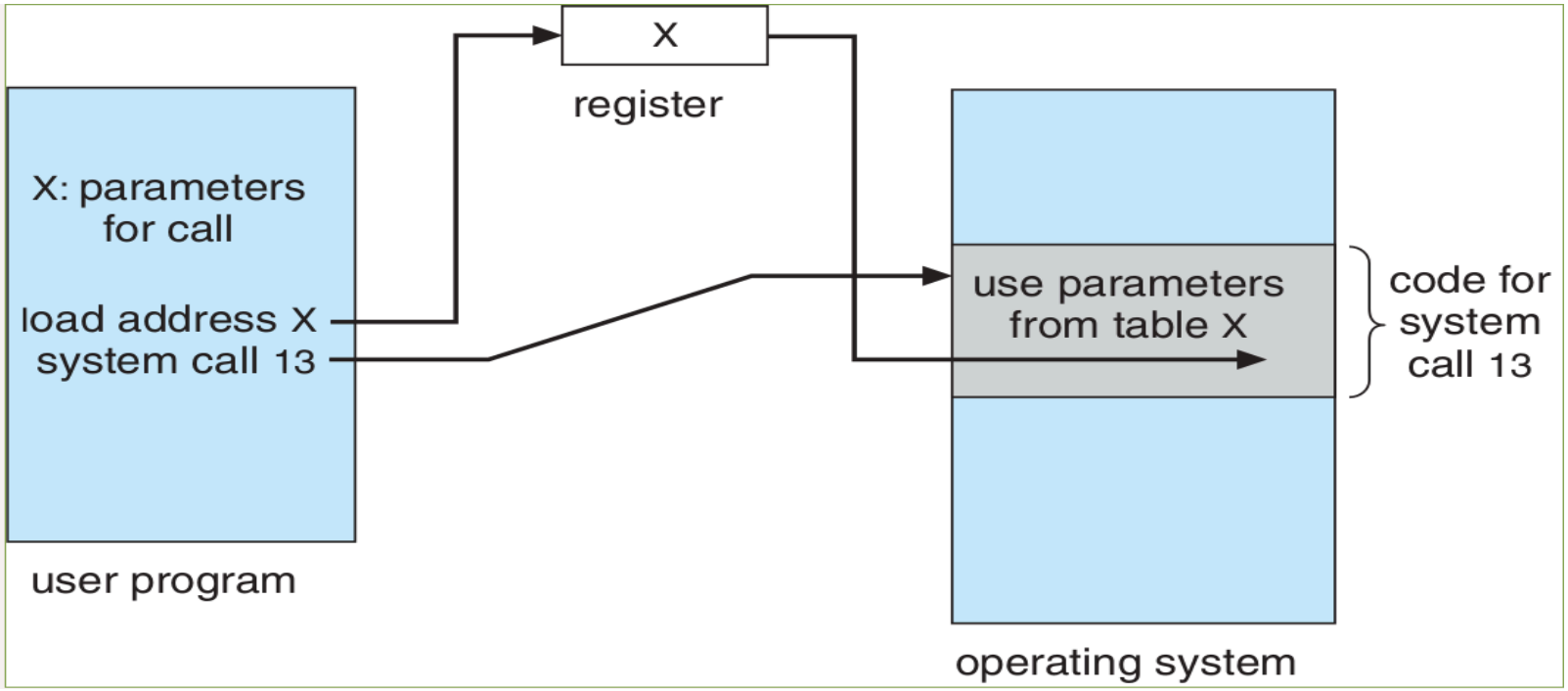
System Call


user process
user process executing
calls system call
return from system call
user mode
(mode bit = 1)
kernel
trap
mode bit = 0
return
mode bit = 1
execute system call
kernel mode
(mode bit = 0)

C System Call

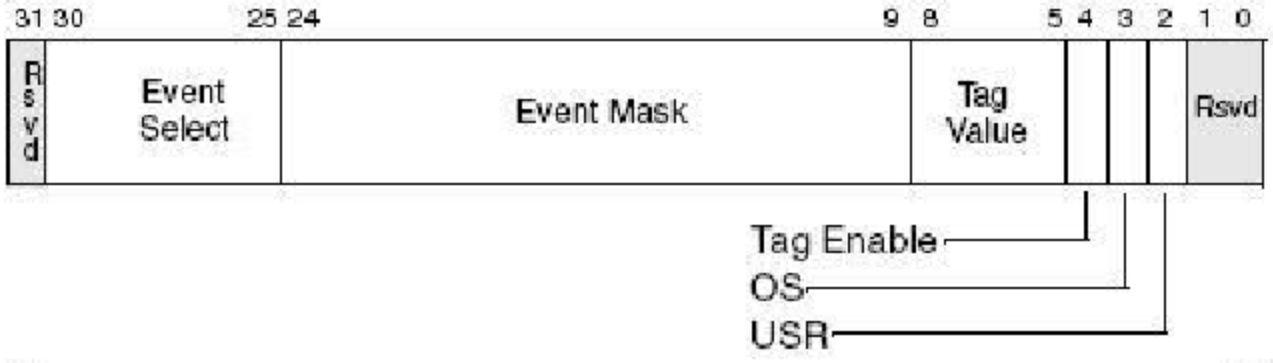


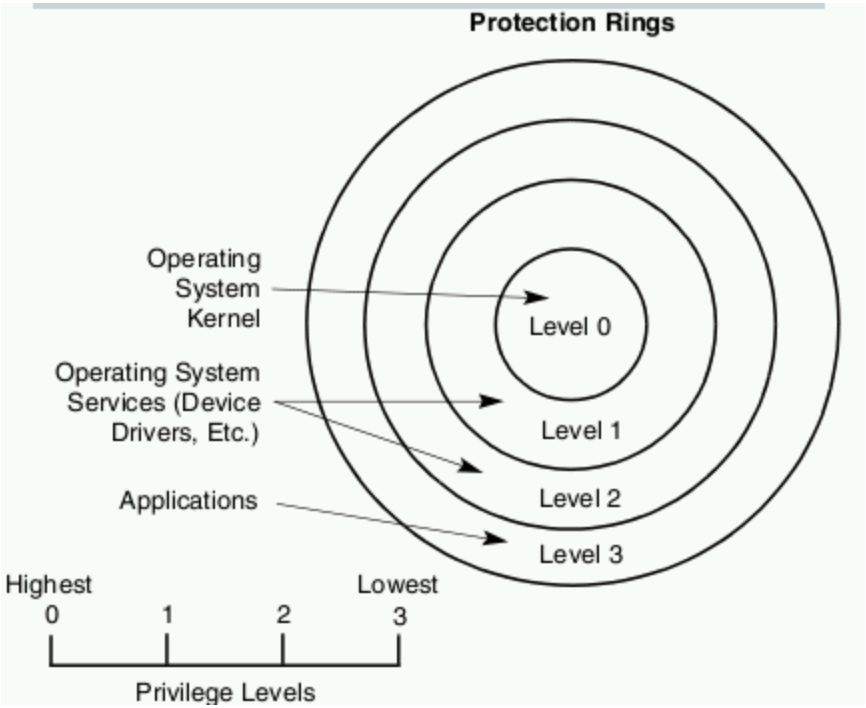
Simple Parameters



- kernel mode
- user mode

Pentium 4 (ESCR)

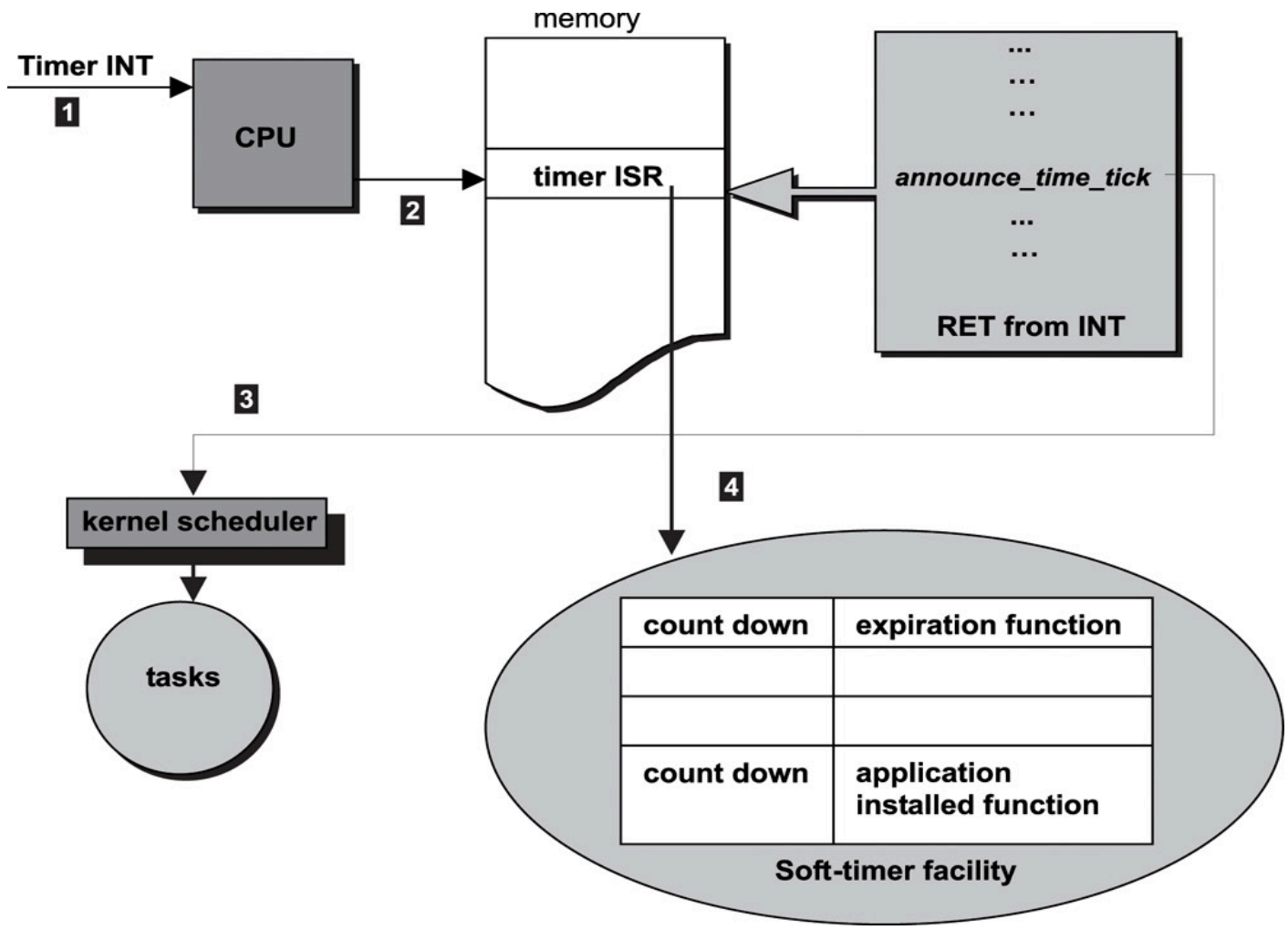



Protection Rings
Operating System Kernel
Operating System Services (Device Drivers, Etc.)
Applications
Level 0
Level 1
Level 2
Level 3
Highest
Lowest
0
1
2
3
Privilege Levels

CPU protection

Timer interrupt

Cpu Scheduler



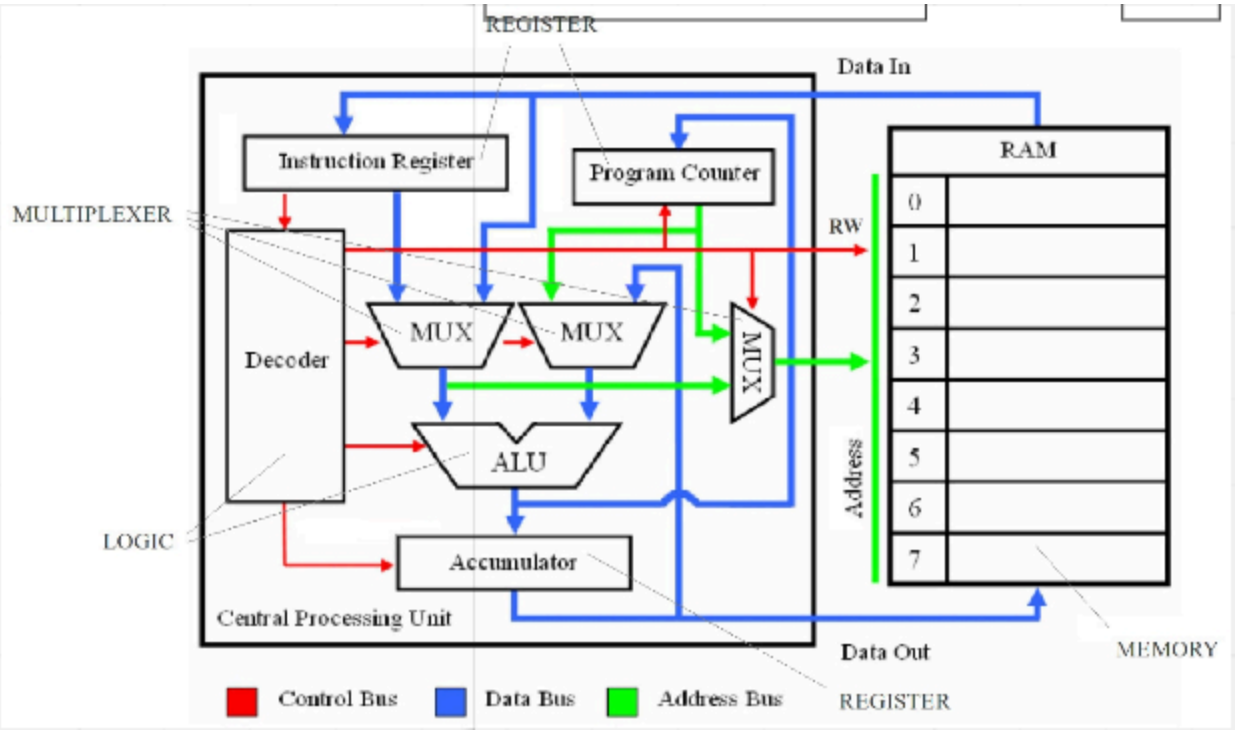
YIC110 - Multiprogramming

Function call

- cons of BSA
    - No recursion
    - No explicit data transfer

Stack From end

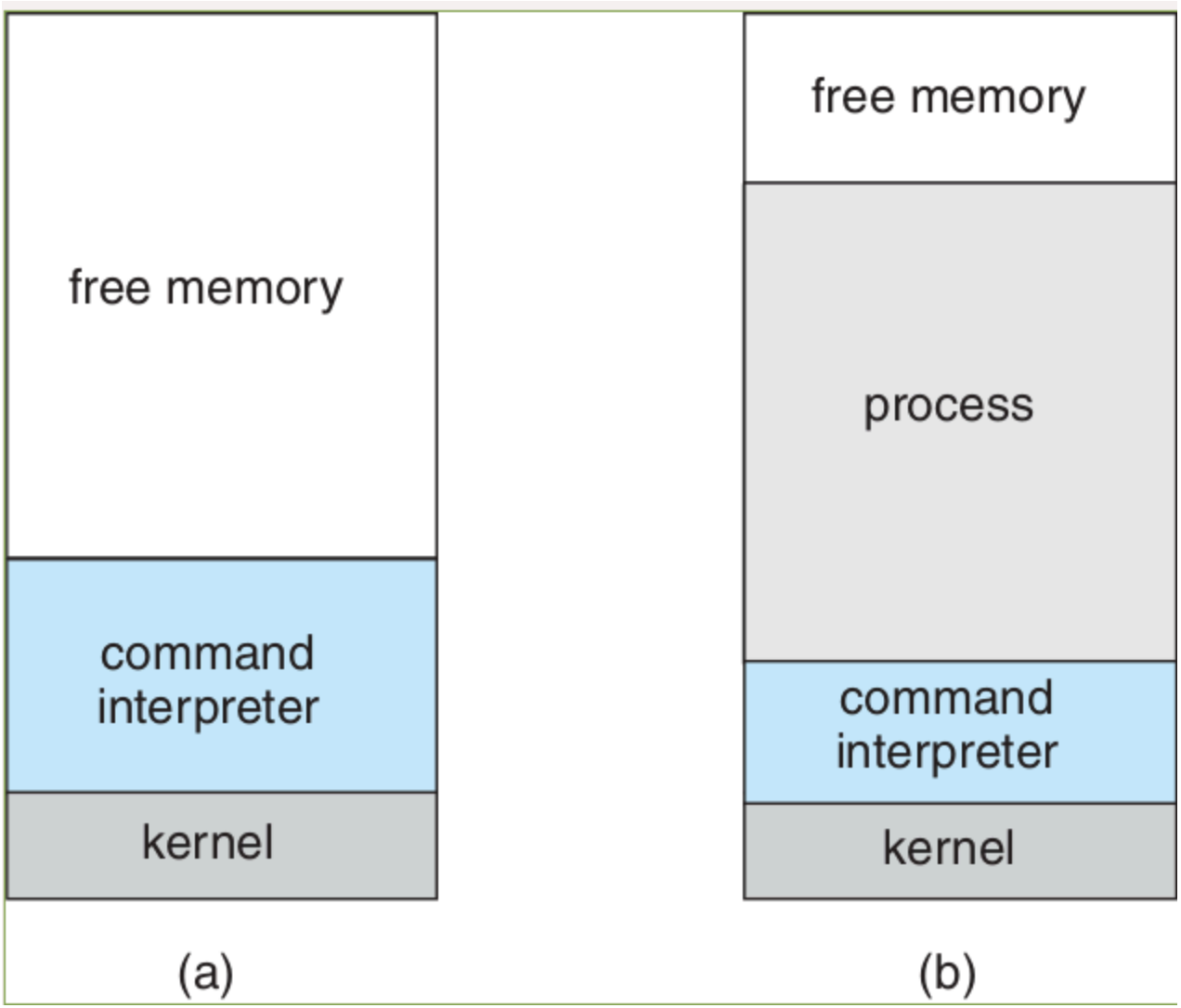
Call, Ret



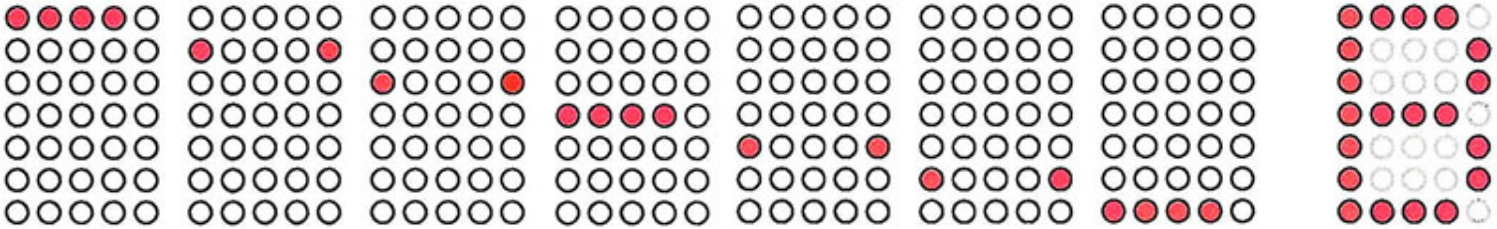
- [Assembly Slides]

YIC120 - Adding Keyboard & Disk

- terminal (command prompt)
- batch system
- interactive system

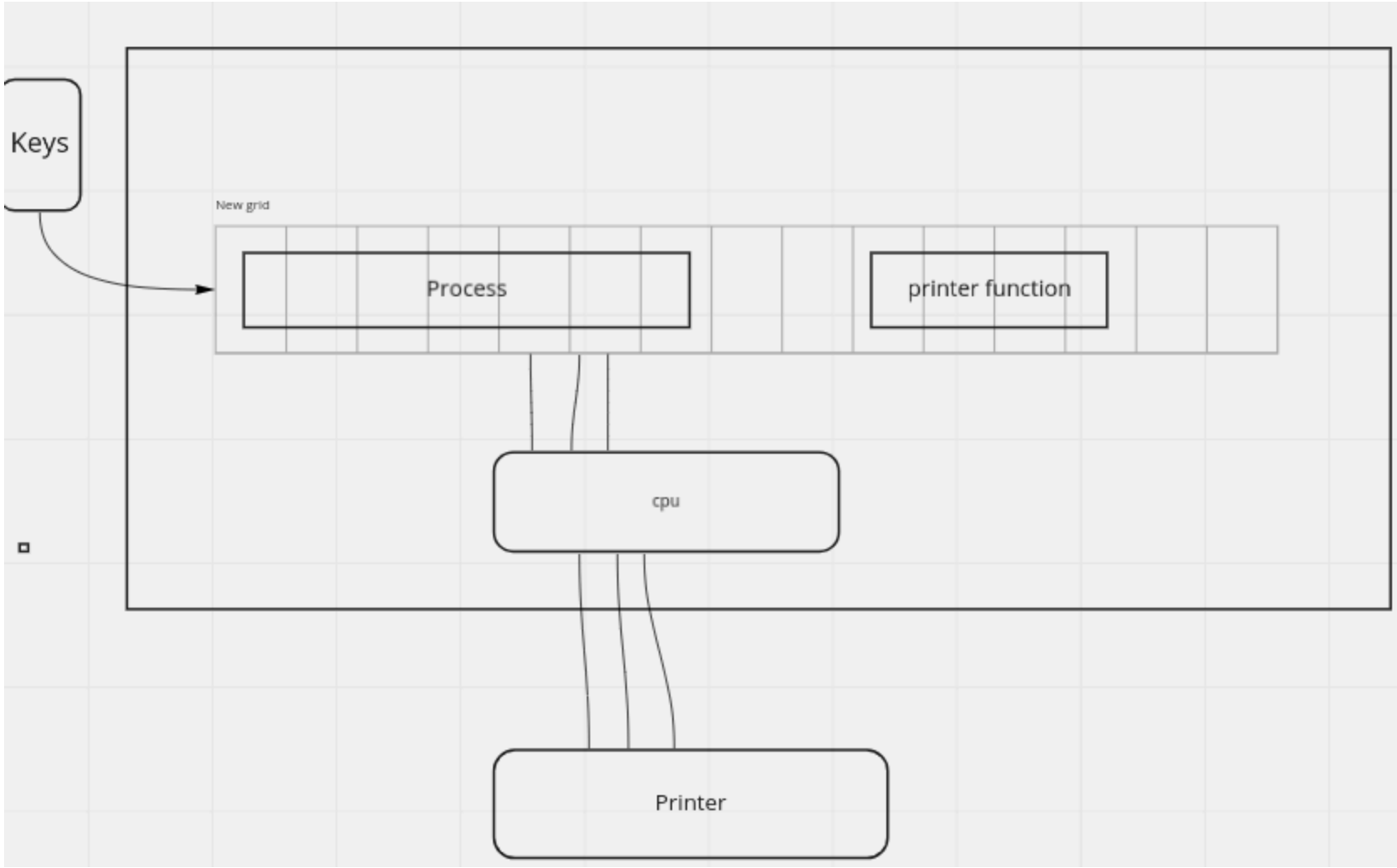

free memory
command interpreter
kernel
(a)
free memory
process
command interpreter
kernel
(b)

When a controller rapidly turns on LEDs in one row at a time

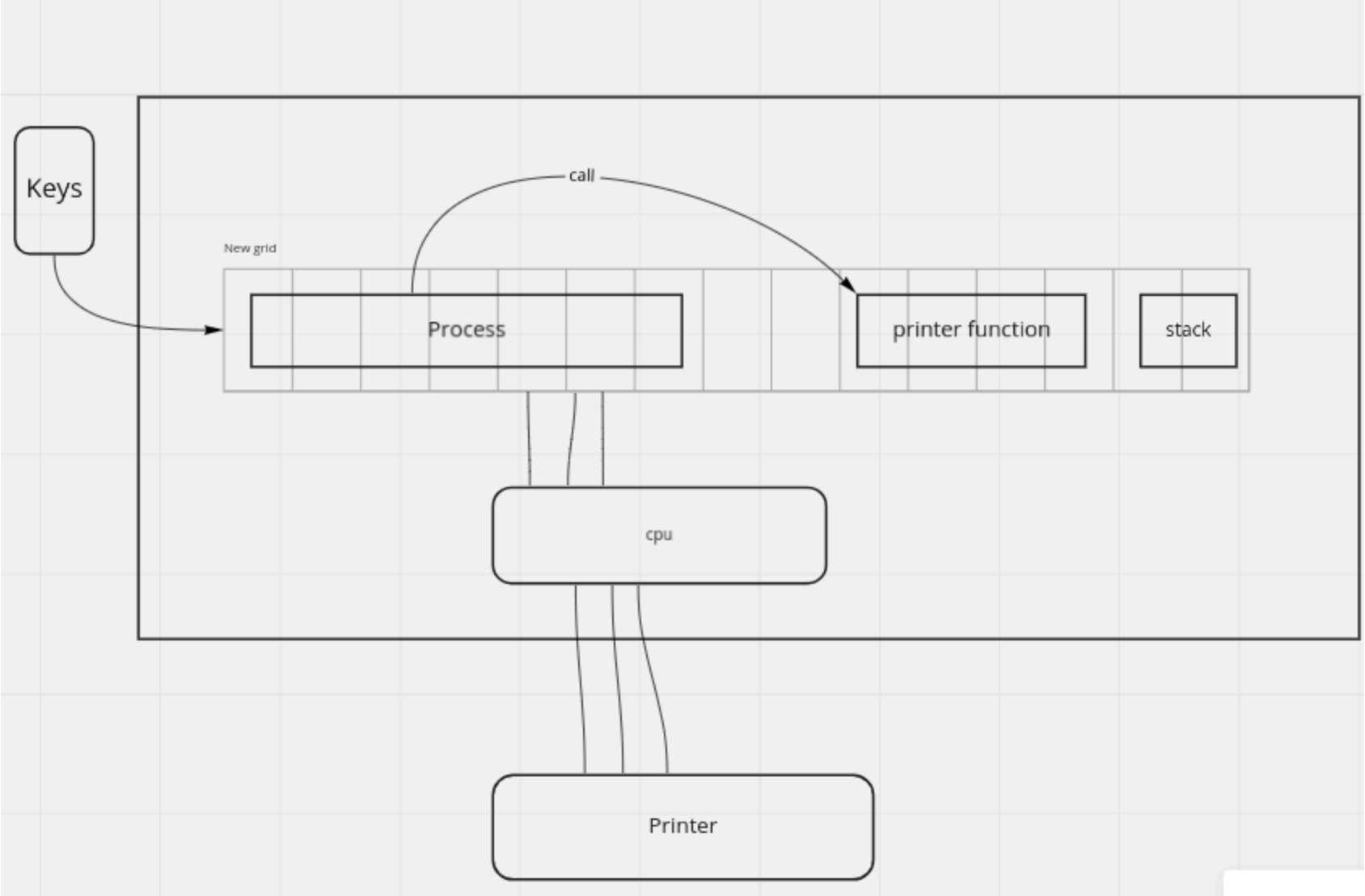


https://www.nutsvolts.com/magazine/article/create-an-led-sign-controller

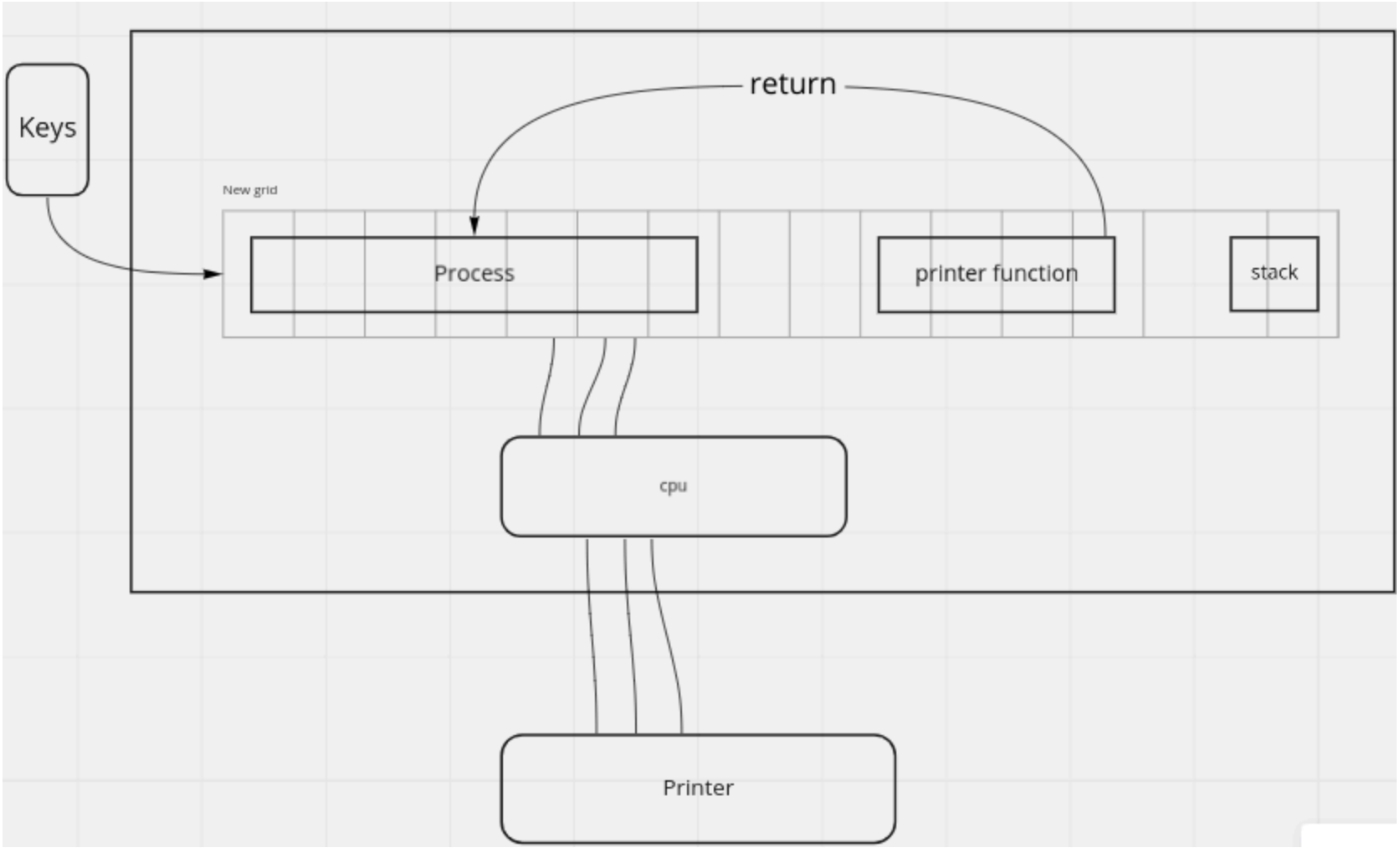
Process along Printer Function



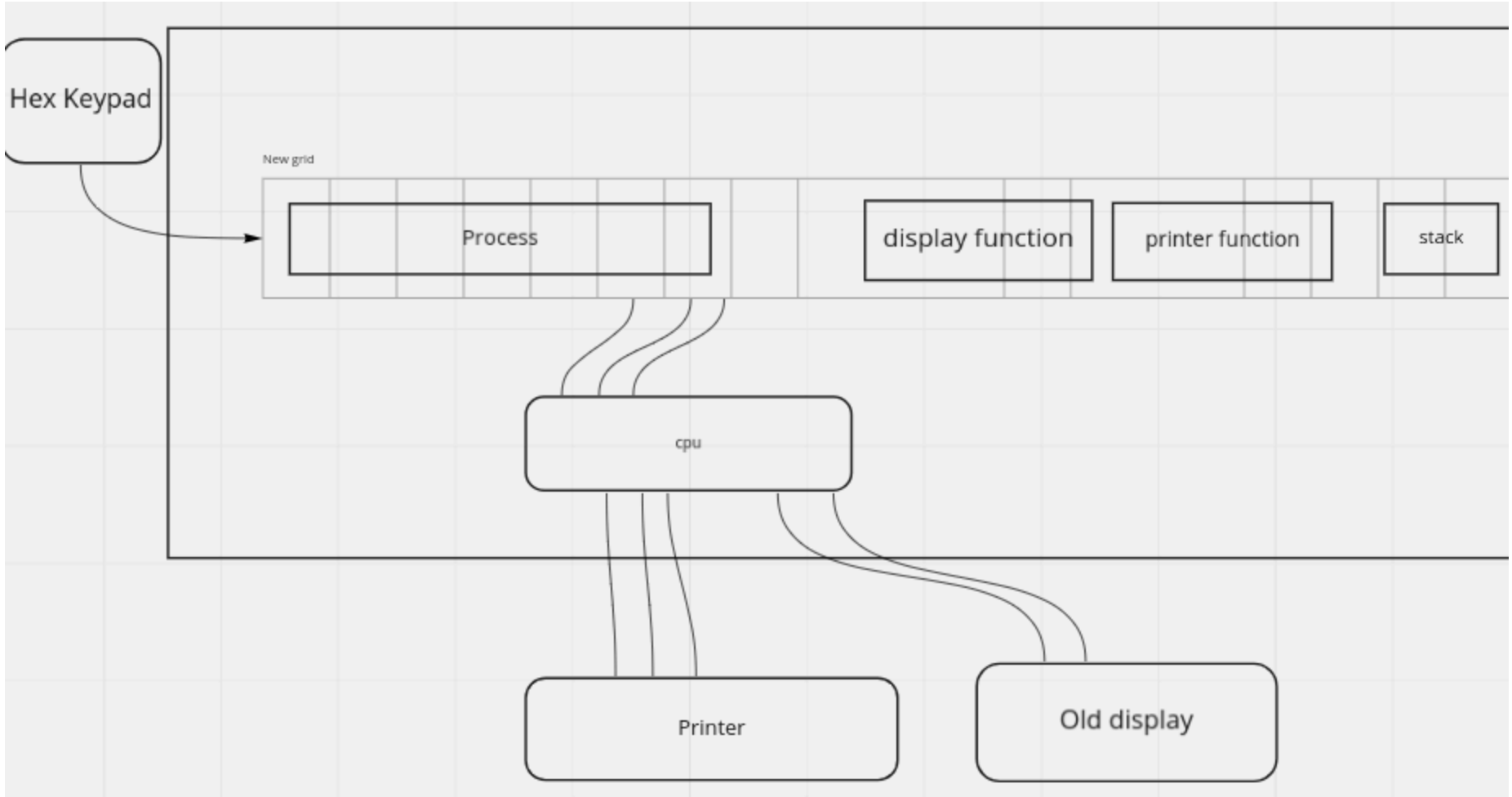
Jump to Printer Procedure



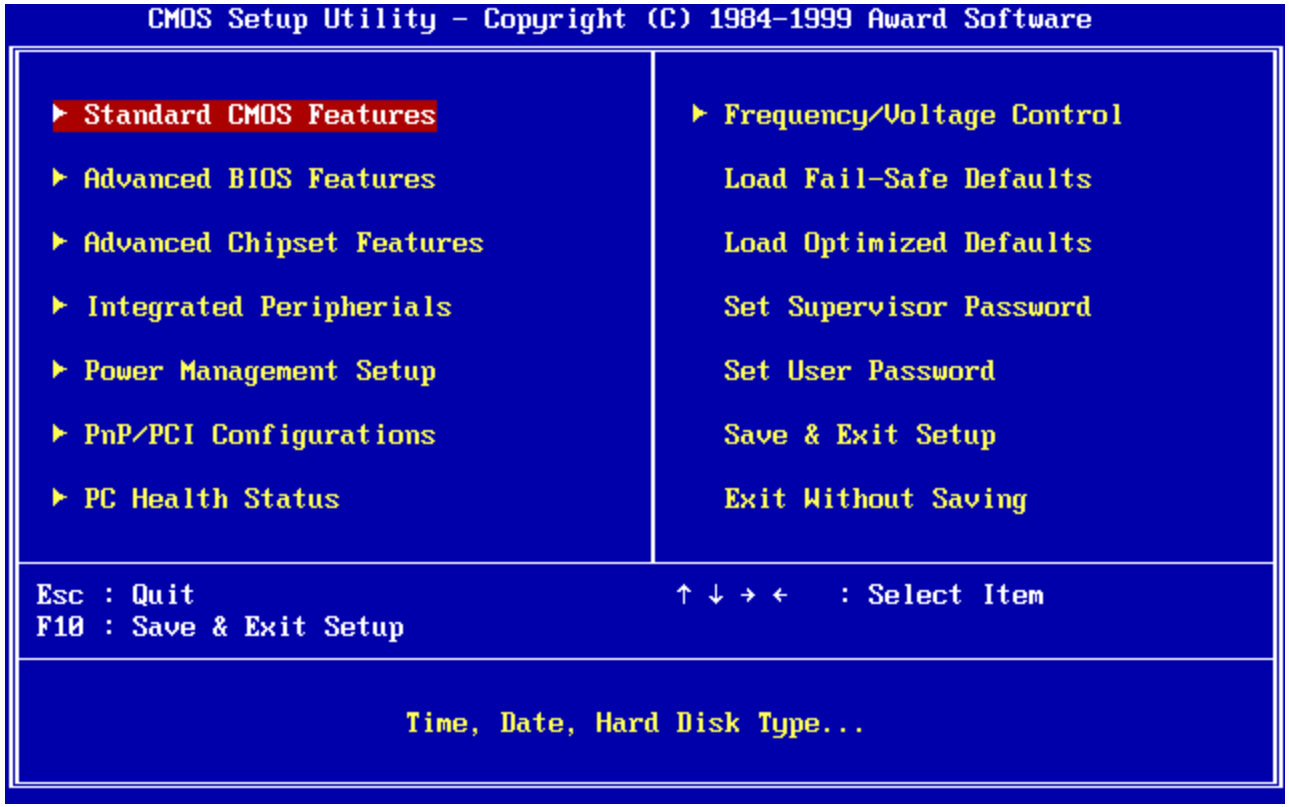
Return from Printer Procedure

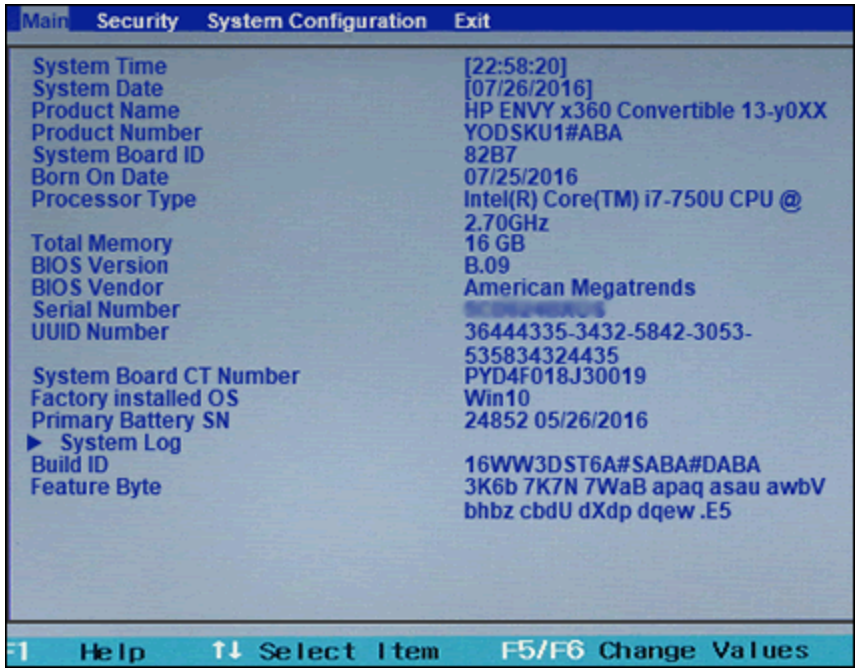


Display and Printer Procedure

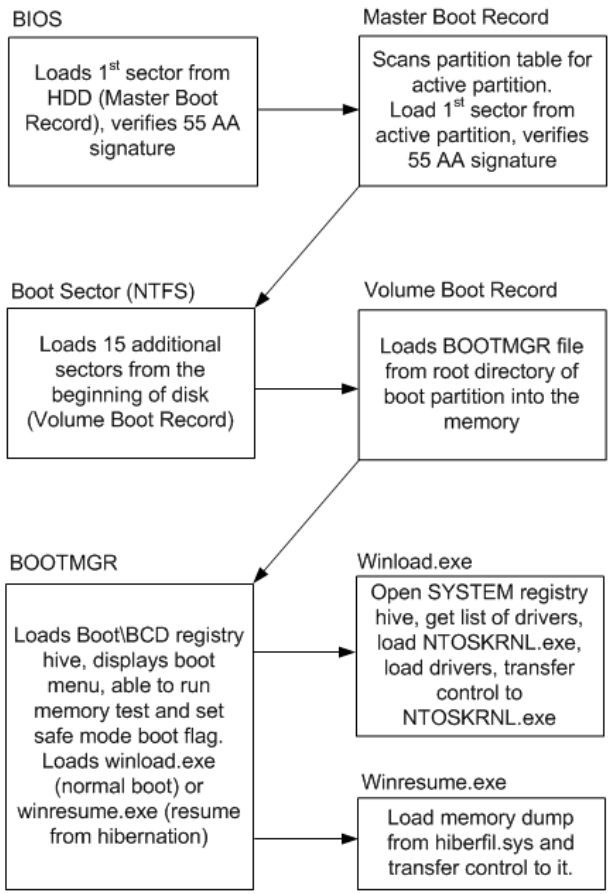


BIOS

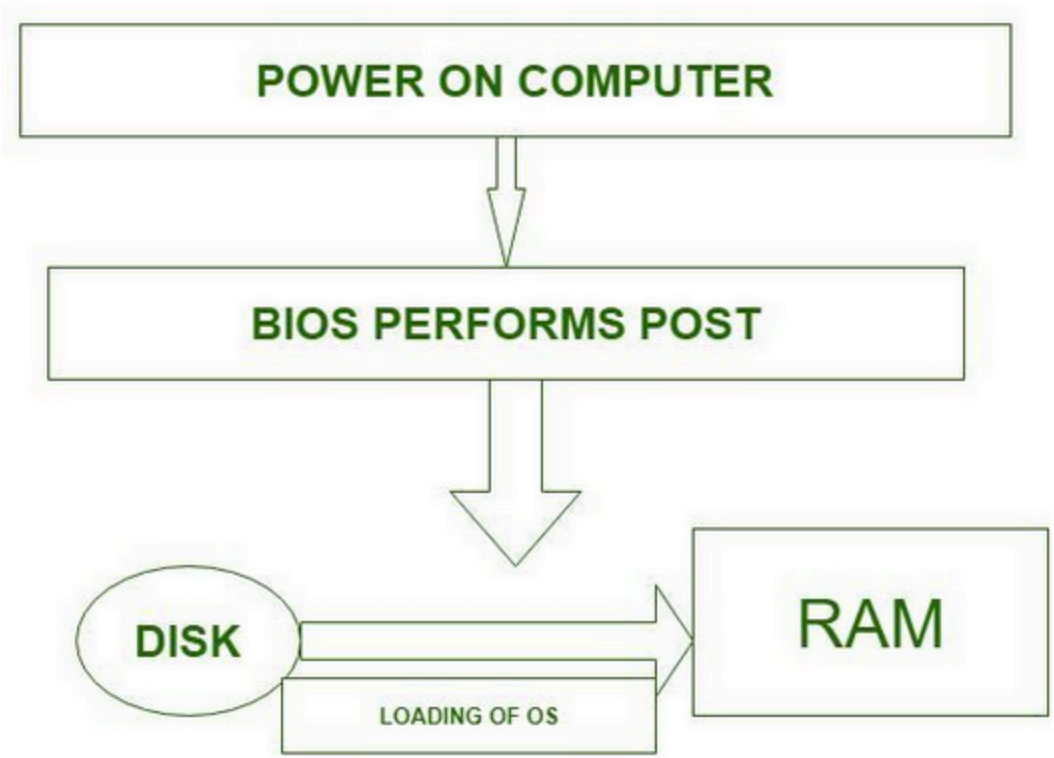

CMOS Setup Utility - Copyright (C) 1984-1999 Award Software
Standard CMOS Features
Advanced BIOS Features
Advanced Chipset Features
Integrated Peripherials
Power Management Setup
PnP/PCI Configurations
PC Health Status
Frequency/Voltage Control
Load Fail-Safe Defaults
Load Optimized Defaults
Set Supervisor Password
Set User Password
Save & Exit Setup
Exit Without Saving
Esc : Quit
F10 : Save & Exit Setup
↑ ↓ → ← : Select Item
Time, Date, Hard Disk Type...


Main Security System Configuration Exit
System Time [22:58:20]
System Date [07/26/2016]
Product Name HP ENVY x360 Convertible 13-y0XX
Product Number YODSKU1#ABA
System Board ID 82B7
Born On Date 07/25/2016
Processor Type Intel(R) Core(TM) i7-750U CPU @ 2.70GHz
Total Memory 16 GB
BIOS Version B.09
BIOS Vendor American Megatrends
Serial Number
UUID Number 36444335-3432-5842-3053-535834324435
System Board CT Number PYD4F018J30019
Factory installed OS Win10
Primary Battery SN 24852 05/26/2016
System Log
Build ID 16WW3DST6A#SABA#DABA
Feature Byte 3K6b 7K7N 7WaB apaq asau awbV bhbz cbdU dXdp dqew .E5
F1 Help ↑↓ Select Item F5/F6 Change Values

Boot sequence



```
Ubuntu 8.04, kernel 2.6.24-16-generic
Ubuntu 8.04, kernel 2.6.24-16-generic (recovery mode)
Ubuntu 8.04, memtest86+
Other operating systems:
Windows Vista/Longhorn (loader)

Use the ↑ and ↓ keys to select which entry is highlighted.
Press enter to boot the selected OS, 'e' to edit the
commands before booting, or 'c' for a command-line.

The highlighted entry will be booted automatically in 4 seconds.
```


POWER ON COMPUTER
BIOS PERFORMS POST
DISK
RAM
LOADING OF OS

- IEEE Std 1275 1994 Standard for boot initialization
- https://openfirmware.info/Welcome_to_OpenBIOS
- https://github.com/openbios
- https://github.com/openbios/openbios

Context Switch

END